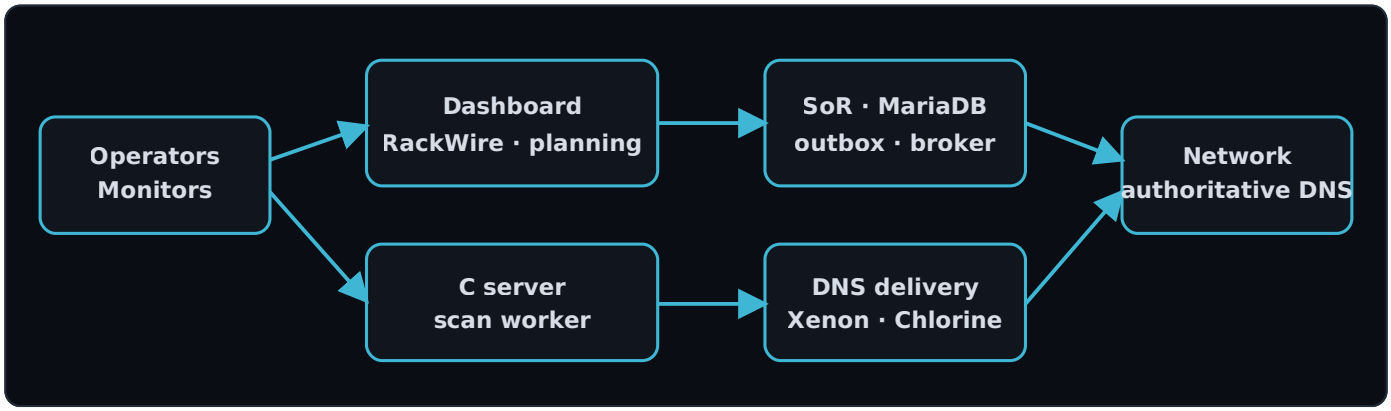
Status & Operations Guide

SolariNet platform · operator / developer handbook · current status 22 October 2026

This guide summarizes system operation and retains the Phase 7 implementation notes below. It is self-contained with inline CSS, JavaScript, and SVG. The dated status card is authoritative for the current integration candidate; Phase 7 counts and claims below are historical snapshots. Follow the linked stabilization workflow and baseline for current release gates.

Current platform state · 22 October 2026

The integration candidate is not released: branch `integrate/stability-20261020` at `789b1d3` is 38 commits ahead of Forgejo and GitHub `main`, whose refs match at `5e8d688`. The realtime Discovery retry, payload-bound idempotency, SoR ingest acceptance, CI, and review documentation are committed on both remotes. GitHub PR #37 run `39715155684` passed all 15 jobs, including the authenticated scanner/RackWire browser flow. Scanner-ingest still has uncommitted manual and validation updates. The public review branch is open as GitHub draft PR `#37`; no main branch merge or release tag exists. The Xenon backup utility is deployed as a local-only recovery path.



- **RabbitMQ:** Benzene runs RabbitMQ `3.13.7` in rootful Podman using the floating `rabbitmq:3-management` tag. The 22 October check found it fully booted with no alarms; each of four `solari` queues had one consumer and zero ready/unacknowledged messages, with seven client connections. Xenon emitter, apply, reconciliation, and notification services were active. Upgrade and recovery remain release gates.

- **Mosquitto:** Benzene's Mosquitto `2.1.2` had a rapid config-open restart loop. With no MQTT clients connected, it was recreated using the same ports and persistent mounts, with a read-only config mount. The password file is owned by UID/GID `1883:1883`, mode `0700`. The replacement runs with zero restarts; the failed container is retained for incident analysis. The original mount failure cause remains unknown. Authbroker is not deployed on Xenon: its unit, virtual environment, and config are absent, so authenticated MQTT client behavior remains unverified. See the Mosquitto deployment guide for the pinned image, secret-free config example, and verification steps.
- **Backups:** Cesium's 21 October SoR dump restored successfully into a disposable MariaDB instance and all 41 tables passed checks. On 22 October, the Xenon runner wrote a 56.4 MB root-only archive and the exact artifact restored into scratch MariaDB; all 29 base tables passed checks. Xenon's nightly timer is enabled for a local root-NVMe target. This shares Xenon's host failure domain. NAS-X provisioning, permissions, and an off-host restore remain unverified.
- **DNS:** Khadas VIM4 chlorine is active as an authoritative-only secondary for 109 zones. Steel at `10.0.0.11` is unreachable; its ZimaBlade tertiary DNS role awaits console redeployment.

- **Network discovery:** operators can start a bounded asynchronous scan; the native worker records live results as reviewable Discovery candidates, which remain separate from physical inventory until explicitly handled. The dashboard polls worker progress every second and refreshes candidate rows every 2.5 seconds while a scan runs, so new hosts appear before completion. RackWire launches the same live Discovery workflow and receives candidate handoff. The native-worker browser acceptance scanned the directly connected `10.0.0.0/24`, processed all 254 addresses, and recorded 14 candidates in a disposable database; the local listener appeared in Discovery before scan completion. The accepted path polls live worker status, refreshes candidates during execution, and supports reviewing and ingesting discovered candidates into inventory. A 22 October combined scan-and-ingest acceptance processed 254/254 addresses, found 12 candidates, observed the listener candidate while the worker was running, retried a lost response without duplicating the job, and adopted `10.0.0.20` into monitoring and the SoR. It used separate disposable stores; no production database was changed. A separate UI acceptance verified candidate refresh before its controlled scan fixture reaches a terminal state. Larger-range throughput and candidate deployment remain unverified. A read-only Xenon check found the standalone RackWire discovery service and Apache health proxy returning HTTP 200, while the dashboard's unauthenticated discovery endpoint returned HTTP 401 as expected. The Xenon-served dashboard bundle differs from this candidate, so those checks do not verify its authenticated real-time scan path. The 22 October scratch-database browser acceptance also exercised the full operator ingest step: it dropped the response after durable scan admission, retried with the same idempotency key, observed one live candidate, adopted it through the Monitor dialog, and verified one monitoring asset plus the mapped address on a SoR entity. The test used separate disposable monitoring and SoR schemas. The combined direct-route `/24` run verified this whole path. Adoption creates a monitored host and SoR host/address; it does not create a physical asset. Physical assets enter through the scanner-ingest preview/apply flow. Full monitoring CTest passed 36/36 against a disposable MariaDB migrated through 033; the separate loopback transport test passed. Hosted run `39715155684` passed all 15 jobs, including the scanner import and scan-to-SoR browser flows. Cross-lab review and the authenticated Xenon browser path remain pending.

- **Inventory:** scanner export, dashboard import, and transactional SoR paths are implemented and pass scratch-database tests. No scanner rows have been imported into the live SoR. The authenticated browser E2E loaded the launched app's exact two-item/two-asset export, previewed and applied it to a disposable SoR, read both assets back, and carried one through RackWire profile setup, barcode resolution, and plan save. It explicitly mapped source locations to duplicate scratch rows; this does not resolve which of the two duplicate root “Painted Fern” sites (IDs `9103`, `9104`) “Bin #5” and “Nook” should map to. The standalone scanner-ingest app has a local Git repository but no configured release remotes; GitHub and Forgejo lookups found no published `scanner-ingest` repository, so its release ownership/path is unresolved.
- **UniFi topology:** a new GET-only discovery command now reads all network pages and details, validates gateway/subnet/DHCP relationships, and emits a canonical snapshot without writing to UniFi or the SoR. The live Default site currently reports only VLAN 1, `10.0.0.0/8`, gateway `10.0.0.1`, and DHCP `10.1.0.26–10.7.161.117`. This conflicts with the project’s segmented network model. The command requires HTTPS and either a trusted controller CA or an explicit insecure-TLS opt-in. No topology import or SoR write is enabled; reconcile this source state before designing the transactional importer. A read-only SoR query returned 16 active /24s across eight named segments; all have VLAN and gateway unset. The UniFi /8 overlaps all 16 and matches none exactly. The cause and source authority are unresolved.
- **Release gates:** authenticated browser acceptance now passes against disposable SoR and broker fixtures, including scanner import and RackWire iframe save. Staged production browser verification, cross-lab review, production migration/rollback, and application deployment remain open. The root review branch has a GitHub draft PR `#37`; RackWire's public GitHub mirror now has main and integration branches plus draft PR `#1`. RackWire's new independent GitHub Actions workflow passed `45/45` backend tests and four runtime checksum checks in run `39704732584`; root CI also recursively checks out its pinned gitlink. Scanner-ingest at `~/Code/scanner-ingest` remains a local repository without remotes or matching GitHub/Forgejo repositories. Claude CLI reports no authenticated session, so cross-lab review is unavailable. The latest root PR CI run, `39715155684` at code-bearing commit `789b1d3`, passed all 15 hosted jobs. The preceding code-bearing run `39705502734` at `0a322eb` passed the real-time scan/candidate-ingest, scanner/RackWire browser, and release-package checks. During an earlier run, the clean-install snapshot check exposed MariaDB client-version drift: hosted 10.11 emits `SQL_NOTES` wrappers while the 11.8 snapshot uses `NOTE_VERBOSITY`; cross-version normalization now passes in hosted CI. The PRs remain drafts pending independent review and release gates. Root and RackWire remain draft PRs, and no release tags exist.

Current evidence and next gates: Stabilization Baseline, Integration Workflow, and the cross-lab review brief.

Contents · Phase 7 deliverables · Build matrix & options · Running server / client / monitor · Dashboard on-prem setup · API / control-plane discipline · The MCP · Enrollment runbook · Decommission runbook · Retention · CI · Known gaps

Phase 7 deliverables

Topology — deep path (server)

`src/server/serverTopology.c` gained a deep-discovery path that normalizes LLDP-MIB rows into the schema's gear/edge model and assembles the `/api/netgear` view — all pure and SNMP-free in the default build:

- **LLDP-MIB normalization.** One decoded `lldpRemTable` row (`topoLldpMibRecord`) becomes a `serverNetGear` + a `serverLldpEdge` (`topoNormalizeMibRecord`). Gear-kind inference (`topoInferGearKind`) treats the LLDP `SystemCapabilitiesMap` bits as authoritative (AP > router > bridge), falling back to sysName / sysDesc keyword matching when caps are absent. Gear IDs (`topoMibGearId`) are stable and kind-prefixed (`sw-/ap-/gw-/rtr-/nd-`), seeded sysName → mgmtIp → chassisId. A C99-safe case-insensitive substring (`topoContainsCi`) avoids GNU `strcasestr` .
- **Uplink-chain + attached-count assembly.** `topoAssembleNetGear` de-duplicates gear by gearId, counts DISTINCT attached SolariNet nodes per gear (gear-to-gear edges excluded; a node seen on multiple interfaces counts once), and computes uplink depth with a cycle guard (`TOPO_MAX_UPLINK_HOPS`). Returns `ERR_BUFFER_FULL` on an undersized output buffer.
- **Two projections.** `/api/topology` serves `view=monitoring` (from HRW + node data) and `view=network` (from `networkGear` + `lldpEdge` + the assembled uplink chain).

> **SNMP capabilities are optional.** With `SOLARI_WITH_SNMP=ON` , the LLDP topology fetch (`topoSnmpFetchLldp`) walks LLDP-MIB columns with the net-snmp CLI, then feeds the tested normalizer; the IF-MIB poller stores interface counters in `gearInterfaceCurrent` and `gearInterfaceHistory` . Both fetch paths return `ERR_PLATFORM` in the default build. The option is off by default and adds no SNMP library build dependency, but requires `snmpbulkwalk` at runtime. Neither path has been verified against a live network device in this release candidate.

> The IF-MIB walk parser now rejects malformed numeric OIDs, invalid or out-of-range values, and interface names that would otherwise be silently truncated; tests pass with the feature both off and on. A fake CLI integration fixture exercises LLDP column collection, normalization, and nonzero-command failure handling without contacting a device.

MCP — Phase 7 catch-up

The Python / FastMCP server (`integrations/mcp/`) grew from 14 to **26 registered tools**, covering the §5/§6 read surface and a gated, audited control surface. The SQL-query tool also gained a table allow-list. Details in the MCP section.

Operational hardening

Enrollment / decommission scripts (`deploy/enrollment/`), hardened systemd units (`deploy/systemd/`), DB maintenance procedures (`db/maintenance/`), and a GitHub Actions CI workflow (`.github/workflows/ci.yml`). All additive — no C sources or dashboard files were modified to land them.

Build matrix & options

The build is layered: a zero-dependency ISO C99 core that builds anywhere, with heavier capability behind explicit CMake options. All options default `OFF` .

OPTION	DEFAULT	PULLS IN	ENABLES
`SOLARI_BUILD_TESTS`	OFF	(vendored Unity)	the ctest suites
`SOLARI_WITH_IO`	OFF	nng + mbedTLS	the transport layer `solariNet`
`SOLARI_WITH_SQLITE`	OFF	SQLite	the store-and-forward spool
`SOLARI_WITH_JSON`	OFF	cJSON	the JSON config overlay
`SOLARI_BUILD_SERVER`	OFF	libmariadb (+ IO transitively)	solariServer (x86_64 / arm64 only)
`SOLARI_WITH_SNMP`	OFF	net-snmp `snmpbulkwalk` at runtime (none vendored or linked)	IF-MIB interface metrics and LLDP topology walks

Cross / platform toolchains

`cmake/toolchains/` carries `linux-x86_64`, `linux-arm64`, `linux-arm32`, `windows-x86_64`, and `macos-universal`. The server tier is built only on the 64-bit Linux targets; the edge agents build across the matrix.

```
# core only (no deps, builds anywhere)
cmake -S . -B build -DSOLARI_BUILD_TESTS=ON
cmake --build build && ctest --test-dir build --output-on-failure

# full server tier
cmake -S . -B build -DSOLARI_BUILD_TESTS=ON -DSOLARI_BUILD_SERVER=ON \
      -DSOLARI_WITH_IO=ON -DSOLARI_WITH_SQLITE=ON -DSOLARI_WITH_JSON=ON
cmake --build build && ctest --test-dir build --output-on-failure
```

> **Verified in this session.** A clean-from-scratch full-server configure + build under `-Wall -Wextra -Wpedantic` reports **0 warnings, 0 errors**; MariaDB was found via pkg-config and linked. `ctest` : **0 failed across 33 registered tests** (server suites included); the DB-live test self-skips without its scratch-DB settings. The MCP suite reports **81 passed**.

Running server / client / monitor

The supported way to run the three roles is via the hardened systemd units in `deploy/systemd/` (§15.2):

UNIT	ROLE	HARDENING NOTE
`solari-server.service`	solariServer (primary or failover — same binary, lease-decided)	`RuntimeDirectory` provides the `solariCtl` socket dir
`solari-monitor.service`	probe monitor	adds `CAP_NET_RAW` for ICMP probing
`solari-client.service`	edge agent	least-privilege sandbox
`solari-client-watchdog.service`	optional external supervisor of the client-spawned watchdog sibling	defense-in-depth for the §7.3 watchdog

```
install -m0644 deploy/systemd/*.service /etc/systemd/system/
systemctl daemon-reload
systemctl enable --now solari-server.service       # the C2 tier
```

```
systemctl enable --now solari-monitor.service      # probe monitor(s)
systemctl enable --now solari-client.service       # on each monitored host
```

The candidate contains 49 service, timer, socket, and path unit files, including 43 services. The dedicated section-placement test passes across all 43 service files. `systemd-analyze verify` reports 14 missing-executable diagnostics for deployment paths absent from this development checkout; it exits nonzero for those unresolved paths. No directive-section errors were emitted. Start-limit directives on the SoR sync and notification units are in `[Unit]`, where systemd applies them.

Dashboard on-prem setup

The dashboard (Phase 6) is fully on-prem: React / ReactDOM / Babel are vendored under `dashboard/public/vendor/`, fonts and icons are local, and the page makes no external fetches. Serve `dashboard/public/` as the static root and `dashboard/api/` (a PDO-backed PHP service) under `/api` on the same origin. The API needs a PDO DSN to the server's MariaDB (reads) and access to the `solariCtl` Unix socket (control actions). See `PHASE6_REPORT.html` for the route map.

API / control-plane discipline

There are three operator surfaces — the dashboard PHP API, the MCP, and the `solariCtl` Unix socket — and they share one discipline:

- **Reads are direct, control is routed.** Read endpoints project the database. State-changing actions never write the DB from the edge surface; they are handed to the C server (via `solariCtl`), which stays the single arbiter.
- **Destructive actions require an operator token *and* explicit confirmation.** Approve-enrollment (signs a CSR) and decommission (wipes a node) refuse before any network call unless both the operator token and an explicit `confirm=true` are present. Decommission uses a stored SHA-256 digest of a random one-time confirmation token, bound to node, wipe scope, operator, and a ten-minute expiry. The server consumes it and retires the node in one transaction; raw tokens are never written to audit details.
- **solariCtl verbs:** `PING`, `SURVEY`, `APPROVE`/`REJECT`, `PROVISION`, `ADOPT`, `IGNORE`, `CONTROL`, `DECOMMISSION`/`RETIRE`.

The MCP

`integrations/mcp/` is a Python / FastMCP server exposing SolariNet to an AI assistant. It wraps the same REST surface (envelope-unwrapping `RestClient`) and fixed read-only database tools; generic caller-supplied SQL is removed. Clients are injectable for tests. Deps live in a git-ignored `.venv`. **26 tools** are listed by the current candidate CLI.

Read tools (GET, read-only)

Among the §6 reads added this phase: `solarinet_list_discovery`, `solarinet_list_enrollments`, `solarinet_list_builds`, `solarinet_list_segments`, `solarinet_list_netgear`, `solarinet_get_topology_view` (dual-hierarchy `view=monitoring|network`), and a read-only `solarinet_get_config`.

Control tools

- **Non-destructive (POST):** `solarinet_adopt_discovery`, `solarinet_ignore_discovery`, `solarinet_reject_enrollment`, `solarinet_provision` (requires node_id or enr_id).
- **Destructive (gated):** `solarinet_approve_enrollment` (signs CSR) and `solarinet_decommission` (wipe). Both are marked `destructiveHint` and require the operator token *and* `confirm=true`, refusing before any network call — mirroring the dashboard discipline. A `_require_operator_token()` helper enforces the token gate.

Database access boundary

MCP callers cannot provide SQL. Three fixed database tools use internal queries; the DB client enforces an explicit table allow-list (`solarinet_mcp/db.py`) alongside SELECT/WITH-only, anti-stacking, no-OUTFILE checks. `_enforce_table_allow_list()` extracts every `FROM` / `JOIN` target, exempts CTE names declared via `WITH`, and rejects anything unlisted — blocking `information_schema` / `mysql` / `performance_schema` exfiltration. The allow-list covers the §10 base tables plus the new §5 tables (`segment`, `networkGear`, `lldpEdge`, `discovered`, `enrollment`, `buildArtifact`).

```
# run the MCP test suite
cd integrations/mcp && .venv/bin/python -m pytest -q     # 81 passed
```

Enrollment runbook

`deploy/enrollment/solari-enroll.sh` enrolls a node end-to-end, generating key material *locally* and only ever sending a CSR:

1. Generates an EC P-256 key + CSR locally (`--rsa` for an RSA key). The CN encodes `role.fqdn` so the role is the leading DNS label used by server ingest authorization (e.g. `CN=monitor.beryl.akoria.net`); the FQDN is also in the DNS SAN.
2. Submits `{token, role, fqdn, certFp, csrPem}` to the server — over HTTPS (`--server`) or a local `solariCtl` Unix socket (`--ctl-socket`).
3. Handles the `status=pending` reply (awaiting operator approval).
4. On approval, installs the signed cert + CA root, verifies the chain, and stamps `nodeId` into `[identity]` of the node config.

```
# enroll a monitor against the server over HTTPS
deploy/enrollment/solari-enroll.sh --role monitor --fqdn beryl.akoria.net \
    --token "$ENROLL_TOKEN" --server https://server.akoria.net

# generate key + CSR only (no submission) — useful for offline / staged enrollment
deploy/enrollment/solari-enroll.sh --role monitor --fqdn beryl.akoria.net --csr-only
```

Key flags: `--role` , `--fqdn` , `--token` , `--server` / `--ctl-socket` , `--rsa` / `--key-bits` , `--csr-only` , `--conf-dir` / `--config` , `--insecure` , `--force` .

> **Verified:** `--csr-only` produced a real EC key + CSR whose `openssl req -verify` returns "self-signature verify OK" (`subject=CN=monitor.beryl.akoria.net`).

Decommission runbook

The dashboard issues a durable decommission challenge on the first queued request. The second request supplies that same token; the server checks its stored digest, node, wipe scope, operator, and expiry before consuming it and publishing a directive with the challenge generation. The node remains active until an authenticated success result echoes that generation. Expired, mismatched, replayed, or force-cancelled challenges fail. The active server persists delivery attempts and retries from 5 seconds up to a 5-minute interval until acknowledgement or cancellation, including after restart or failover. The client currently has no receiver for this directive, so remote teardown still cannot complete. The privileged request queue also retains the raw confirmation token in its request/result text. Do not treat a

sent directive as a completed wipe. `deploy/enrollment/solari-decommission.sh` remains a separate node-local, operator-driven executor (Handoff §4.2 / §7.3), not an SCP message receiver:

1. Requires a server-issued `--confirm-token` (`TLV_LIFE_CONFIRM`) echoed for audit, plus an interactive phrase confirmation.
2. `--wipe-scope config,certs,spool,logs,unit,all` (`TLV_LIFE_WIPE_SCOPE`) selects what is erased.
3. Stops the service, reports retirement best-effort *before* key deletion, securely erases the scoped material (`shred` when available), then disables / removes the unit.
4. `--dry-run` to rehearse; `--local-force --yes` for an offline or forced teardown when the server is unreachable.

```
# server-driven, audited teardown of a node
deploy/enrollment/solari-decommission.sh --confirm-token "$TOKEN" --wipe-scope all
# rehearse without touching anything
deploy/enrollment/solari-decommission.sh --confirm-token "$TOKEN" --wipe-scope config,certs
--dry-run
```

> **Verified:** scope parsing and a full `--dry-run` exit cleanly (rc=0); both scripts pass `bash -n` .

Retention & history rolling

`db/maintenance/` holds MariaDB stored procedures that operationalize the `TO_DAYS(sampledAt)` RANGE partition scheme from `db/schema.sql` :

- `retention.sql` — `solari_purge_retired` / `solari_purge_history` / `solari_retention_run` : purge `state='retired'` nodes (with FK-cascade children plus explicit `nodeConfig` / `hostHistory`) past the retention window, with a row-level history safety net.
- `roll_history.sql` — `solari_roll_partitions` : REORGANIZE the `p_max` catch-all into monthly `pYYYYMM` partitions ahead of now and DROP-PARTITION months aged past retention, on both `hostHistory` and `probeHistory` . Idempotent via `information_schema.PARTITIONS` guards; dynamic DDL via PREPARE/EXECUTE.

Schedule them from cron / an event scheduler, e.g. `CALL solari_retention_run(...)` nightly and `CALL solari_roll_partitions(...)` monthly.

CI

`.github/workflows/ci.yml` defines ten job groups and the current draft PR runs 15 hosted checks. The local `build-server-asan` CTest run passed **37**/**37**. The first hosted run passed 12 of 15 checks; subsequent runs caught a date-dependent SOA serial test, a missing RackWire mirror, a missing server-binary artifact in the dashboard job, server checkout omitting the RackWire submodule, and clean-install schema formatting between MariaDB 10.11 and 11.8 dump clients and release packaging initially included subproject CI metadata; the packager now excludes that non-runtime tree. Run `39705502734` passed all 15 hosted checks, including the clean-install ledger and schema snapshot comparison. RackWire's own workflow, run `39704732584`, independently passed its 45 backend tests and four runtime checksum checks.

- `core` — the default zero-dependency build and CTest suite.
- `server` — full server build with ASan/UBSan, database migrations, enrollment and lease failover tests, scan-worker and adoption integration, and browser acceptance.
- `dashboard` — PHP/API and authorization tests, migration contracts, RackWire APIs, scanner-to-RackWire browser flow, and deployment-package checks.
- `rackwire-browser` — standalone offline, accessible, responsive planner flow.
- `python-integrations` — MCP, alertbridge, UniFi, authbroker, notifications, and Opie tests.
- `sorsync` — SoR outbox delivery across RabbitMQ releases and an upgrade/recovery rehearsal.
- `protocol-fuzz` — coverage-instrumented SCP parser fuzz smoke tests.
- `panel` — firmware, daemon, provisioning-tool, and dashboard parity suites.
- `cross` — Linux ARM, Windows, and other cross-target builds with qemu-user smoke runs where available.
- `macos` — universal build and native core tests.

Loopback transport staging is best-effort, mirroring `bootstrap-xenon.sh`. The RackWire and dashboard jobs check out submodules recursively; the RackWire GitHub mirror is available. The RackWire repository now runs its own backend and runtime-integrity checks on pull requests. Other CI results remain local or unverified as stated in their gates.

Known gaps & deferred items

- **Production MariaDB adoption remains unverified.** Disposable MariaDB integration tests have exercised the migration ledger and selected SoR transaction paths. They do not prove migration adoption against current production databases, backups, or restore procedures. Review the current stabilization baseline before operational changes.
- **Production browser / PHP-FPM acceptance remains open.** Authenticated browser tests now exercise multi-session freshness, scanner import, and embedded RackWire save against local PHP and disposable MariaDB fixtures. These do not establish behavior after deployment with PHP-FPM, production authentication, the live MariaDB, and `solariCtl`. The retired SSE endpoint is covered by a route test; a durable event bridge remains future work.
- **Live SNMP device acceptance remains open.** Both LLDP topology fetching and IF-MIB interface polling are implemented behind `SOLARI_WITH_SNMP` and use the net-snmp CLI at runtime. Parser and merge tests pass, and the server builds with the feature both off and on. No live switch or router query has verified credentials, device OID support, data freshness, or persisted results in this release candidate.
- **Production systemd and hosted CI acceptance remain open.** The section-placement check passes for 43 service files. Structural verification of 49 candidate unit files still returns 14 missing-executable diagnostics for binaries or virtual environments absent from this workspace. Enrollment and decommission scripts passed ShellCheck with sourced-file analysis. The GitHub workflow parses locally but has not run on GitHub from this session.

Status card updated 22 October 2026. Sections below preserve the Phase 7 implementation snapshot and its test counts. For current test results and release gates, use the linked `SolariNet_Stabilization_Baseline_2026-10-20` and `SolariNet_Stabilization_Integration_Workflow_2026-08-09`. This guide does not declare the integration candidate ready for release.